Kerberos Performance Monitoring & Anomaly Detection

Complete Guide

Author: Emerson
Version: 1.0
Date: February 2025

📊 Overview

This system monitors KDC (Key Distribution Center) performance and detects anomalies that may indicate:

- **Active attacks** (brute force causing CPU/memory spikes)
- **Infrastructure problems** (memory leaks, disk issues)
- **Capacity issues** (need to scale)
- **Service degradation** (before total failure)
- **Misconfigurations** (database locks, network problems)

🎯 What It Does

Metrics Collected

Authentication Metrics:
- AS-REQ count (initial authentication requests)
- TGS-REQ count (service ticket requests)
- Success/failure rates
- Error rate percentage

Performance Metrics:
- Average response time
- P95 response time (95th percentile)
- P99 response time (99th percentile)

System Metrics:
- CPU usage (system-wide and KDC process)
- Memory usage (system-wide and KDC process)
- Disk I/O (read/write rates)
- Network traffic (packets in/out)

KDC Process Metrics:
- Process CPU percentage
- Process memory consumption
- Thread count
- Open file descriptors
- Active connections

Error Metrics:
- Overall error rate
- Timeout count
- Failed authentication patterns

Anomaly Detection

Statistical Anomaly Detection:
- Establishes baseline from historical data (default: 24 hours)
- Uses mean + 2σ (standard deviations) for threshold
- Detects when metrics deviate significantly from normal

Absolute Threshold Detection:
- Critical thresholds for key metrics
- CPU > 90%, Memory > 90%, Response time > 1s, etc.
- Immediate alerts for dangerous conditions

AI-Enhanced Analysis:
- Uses Claude to understand anomalies in context
- Correlates multiple anomalies
- Identifies root causes
- Provides prioritized remediation steps

🚀 Quick Start

Installation

```
# Install dependencies
pip install psutil anthropic colorama

# For Kerberos client tools
apt install krb5-user

# Copy the script
cp kerberos_performance_monitor.py /usr/local/bin/
chmod +x /usr/local/bin/kerberos_performance_monitor.py
```

Basic Usage

```
# Run once (single check)
sudo python3 kerberos_performance_monitor.py --once

# Continuous monitoring (60 second intervals)
sudo python3 kerberos_performance_monitor.py

# With AI analysis
export ANTHROPIC_API_KEY="your-key-here"
sudo python3 kerberos_performance_monitor.py --ai

# Custom interval and KDC
sudo python3 kerberos_performance_monitor.py \
    --kdc 192.168.88.20 \
    --interval 30 \
    --baseline-hours 48
```

Example Output

```
============================================================
Kerberos Performance Monitor v1.0
============================================================
KDC: 192.168.88.20
Interval: 60s
Baseline: 24h
```

```
AI Analysis: Enabled
============================================================

[+] Found KDC process: PID 1234
[+] Baseline established from 24h of data

[Cycle 1] 2025-02-07 10:15:30

Current Metrics:
  Authentication:
    AS-REQ: 45 (Success: 42, Failed: 3)
    TGS-REQ: 123 (Success: 121, Failed: 2)
    Error Rate: 3.57%
  Performance:
    Avg Response: 45.23ms
    P95 Response: 89.12ms
    P99 Response: 145.67ms
  System:
    CPU: 12.3%
    Memory: 34.5%
    KDC CPU: 8.7%
    KDC Memory: 234.5MB
    Active Connections: 67

✓ All metrics within normal range
```

Anomaly Detection Example

```
============================================================
ANOMALIES DETECTED: 3
============================================================

[HIGH] as_req_count
  as_req_count increased by 245.0% (current: 345.00, baseline: 100.00)
  Possible causes:
    • Brute force attack in progress
    • Legitimate authentication spike (shift change, system startup)
  Recommended actions:
    → Check KDC logs for failed authentication patterns
    → Implement rate limiting if not present

[CRITICAL] error_rate
  error_rate increased by 450.0% (current: 22.50, baseline: 5.00)
  Possible causes:
```

```
    • Invalid credentials being used (attack or misconfiguration)
    • KDC database corruption
  Recommended actions:
    → Review recent failed authentication attempts
    → Check client configurations

[MEDIUM] avg_response_time_ms
  avg_response_time_ms increased by 180.0% (current: 420.00, baseline: 150.00)
  Possible causes:
    • KDC database lock contention
    • High system load (CPU/memory)
  Recommended actions:
    → Check KDC system resources (CPU, memory, disk)
    → Review KDC database performance

================================================================
AI Analysis
================================================================
Root Cause: Coordinated brute force attack causing KDC overload
Risk: HIGH - Authentication service degradation in progress
Priority: 1. Block attacking source IPs, 2. Scale KDC resources
================================================================
```

📈 Understanding Metrics

Authentication Rate

Normal: 10-100 AS-REQ per minute (varies by organization)
High: 500+ AS-REQ per minute
Attack Indicator: Sustained high rate with high failure rate

What to look for:
- Sudden spike in AS-REQ count
- High failure rate (>10%)
- Pattern: same source IP, multiple usernames

Response Time

Normal: 10-100ms
Slow: 200-500ms
Critical: >1000ms

Causes of slow responses:
- Database contention
- High CPU/memory
- Network latency
- Disk I/O bottleneck

Error Rate

Normal: <5%
Elevated: 5-10%
Critical: >10%

Common causes:
- Failed authentications (wrong password)
- Clock skew (time sync issues)
- Database corruption
- Network problems

CPU Usage

KDC Process Normal: 5-20%
KDC Process High: 50-80%
KDC Process Critical: >90%

System-wide should remain <80%

Memory Usage

KDC Process Normal: 100-500MB
KDC Process High: 1-2GB
KDC Process Critical: >2GB (potential leak)

Monitor for:
- Gradual memory growth (leak)
- Sudden spike (cache bloat)

🔍 Anomaly Types & Responses

1. Brute Force Attack

Indicators:
- ✓ High AS-REQ count
- ✓ High error rate
- ✓ Same source IP
- ✓ Multiple different usernames

Response:
1. Identify attacking source IP(s)
2. Block at firewall level
3. Enable account lockout policies
4. Review affected accounts
5. Force password resets if needed

2. Resource Exhaustion

Indicators:
- ✓ High CPU/memory
- ✓ Slow response times
- ✓ High connection count

Response:
1. Check system resources
2. Review KDC logs for errors
3. Restart KDC if memory leak
4. Scale horizontally (add KDC replicas)
5. Optimize database

3. Database Issues

Indicators:
- ✓ Slow response times
- ✓ Normal CPU/memory
- ✓ Disk I/O spikes
- ✓ Intermittent failures

Response:
1. Check database integrity
2. Review disk space

3. Optimize database indexes
4. Check for lock contention
5. Consider database maintenance

4. Network Problems

Indicators:
- ✓ Timeouts
- ✓ Variable response times
- ✓ Normal KDC metrics

Response:
1. Check network connectivity
2. Review firewall rules
3. Check DNS resolution
4. Verify routing
5. Test network latency

5. Clock Skew

Indicators:
- ✓ Authentication failures
- ✓ Error logs showing time issues
- ✓ Specific error codes

Response:
1. Verify NTP configuration
2. Check time sync on clients
3. Check time sync on KDC
4. Review allowed clock skew settings

🤖 AI Analysis Features

What AI Provides

1. **Root Cause Analysis**
   - Analyzes multiple anomalies together
   - Identifies underlying cause
   - Distinguishes attack from infrastructure issue

2. **Risk Assessment**
   - Evaluates impact on authentication services
   - Prioritizes based on severity
   - Considers business impact

3. **Correlation**
   - Links related anomalies
   - Identifies cascading failures
   - Shows cause-and-effect relationships

4. **Action Prioritization**
   - Ranks remediation steps
   - Provides step-by-step plan
   - Considers urgency and impact

Example AI Analysis

Scenario: Multiple anomalies detected

Traditional Alert:
- HIGH: as_req_count increased
- HIGH: error_rate increased
- MEDIUM: avg_response_time_ms increased

AI-Enhanced Analysis:

```
{
  "root_cause": "Brute force attack targeting multiple accounts causing KDC overload",
  "correlation": "All anomalies are related - attack is primary cause, others are second
  "risk_assessment": "HIGH - Authentication services degraded but still functional",
  "priority": "1. Block attacking IPs (immediate), 2. Review logs for compromised accoun
  "time_to_impact": "30-60 minutes until severe degradation if unaddressed"
}
```

Value: Instead of 3 separate alerts, you get one clear understanding of what's happening and what to do.

📊 Baseline Establishment

How Baseline Works

1. **Data Collection**
   - Collects metrics every interval (default: 60s)
   - Stores in SQLite database
   - Keeps historical data for analysis

2. **Statistical Analysis**
   - Calculates mean and standard deviation for each metric
   - Uses last N hours of data (default: 24h)
   - Updates periodically (default: every hour)

3. **Threshold Calculation**
   - Upper threshold: Mean + 2σ
   - Lower threshold: Mean - 2σ
   - Captures ~95% of normal variation

Baseline Best Practices

Initial Setup (First 24 hours):
- System will collect data but not alert
- Ensure this period represents normal load
- Avoid anomalies during baseline establishment

Periodic Updates:
- Baseline recalculates every hour
- Adapts to changing usage patterns
- Accounts for daily/weekly cycles

Manual Baseline Reset:
- After major changes (hardware upgrade, config change)
- After attack (to remove attack data from baseline)
- Seasonally (if usage patterns change)

```
# Reset baseline (delete database)
rm performance_monitoring/metrics.db

# Will rebuild from scratch
```

🔧 Configuration

Metric Collection Intervals

```
# Very frequent (high resolution, more CPU)
--interval 30   # 30 seconds

# Standard (balanced)
--interval 60   # 60 seconds (default)

# Infrequent (low overhead)
--interval 300  # 5 minutes
```

Recommendation: 60 seconds for production, 30 seconds during investigation

Baseline Window

```
# Short window (adapts quickly, less stable)
--baseline-hours 12

# Standard (balanced)
--baseline-hours 24  # Default

# Long window (very stable, slow to adapt)
--baseline-hours 168  # 1 week
```

Recommendation: 24 hours for most environments, 168 hours for very stable environments

AI Analysis

```
# Enable AI
--ai --api-key YOUR_KEY

# Or use environment variable
export ANTHROPIC_API_KEY="your-key-here"
python3 kerberos_performance_monitor.py --ai
```

Cost: ~$0.01 per analysis (very inexpensive)
Value: Significantly reduces investigation time

📁 Output Files

Metrics Database

Location: `performance_monitoring/metrics.db`
Type: SQLite database
Contains: All historical metrics

Query examples:

```
-- View recent metrics
SELECT * FROM metrics
ORDER BY timestamp DESC
LIMIT 100;

-- Calculate hourly averages
SELECT
    strftime('%Y-%m-%d %H:00:00', timestamp, 'unixepoch') as hour,
    AVG(as_req_count) as avg_as_req,
    AVG(error_rate) as avg_error_rate
FROM metrics
GROUP BY hour
ORDER BY hour DESC;
```

Anomaly Reports

Location: `performance_monitoring/anomalies_YYYYMMDD_HHMMSS.json`
Type: JSON
Contains: Detected anomalies with context

Example:

```
{
  "timestamp": "2025-02-07T10:15:30",
  "metrics": {
    "as_req_count": 345,
    "error_rate": 22.5,
    "avg_response_time_ms": 420
  },
  "anomalies": [
    {
```

```
      "severity": "HIGH",
      "metric_name": "as_req_count",
      "current_value": 345,
      "baseline_value": 100,
      "deviation_percent": 245,
      "possible_causes": [...],
      "recommended_actions": [...]
    }
  ]
}
```

🔗 Integration

With SIEM (Splunk, ELK)

Forward anomaly JSON files:

```
# Splunk forwarder
[monitor:///path/to/performance_monitoring/anomalies_*.json]
sourcetype = kdc_anomaly
index = security

# Logstash input
input {
  file {
    path => "/path/to/performance_monitoring/anomalies_*.json"
    codec => "json"
    type => "kdc_anomaly"
  }
}
```

With Alerting (PagerDuty, OpsGenie)

Send alerts on CRITICAL anomalies:

```
# Add to kerberos_performance_monitor.py

def send_alert(anomaly: PerformanceAnomaly):
    if anomaly.severity == "CRITICAL":
```

```
        # PagerDuty example
        requests.post(
            "https://events.pagerduty.com/v2/enqueue",
            json={
                "routing_key": "YOUR_KEY",
                "event_action": "trigger",
                "payload": {
                    "summary": f"{anomaly.metric_name} anomaly",
                    "severity": "critical",
                    "source": "kdc_monitor"
                }
            }
        )
```

With Grafana

Visualize metrics from SQLite database:

1. Install SQLite datasource plugin
2. Configure datasource pointing to metrics.db
3. Create dashboards with queries
4. Set up alerts based on thresholds

🎯 Use Cases

1. SOC Operations

Scenario: 24/7 monitoring of authentication infrastructure

Setup:

```
# Run as systemd service
sudo systemctl start kdc-monitor

# AI analysis enabled
# Baseline: 24 hours
# Interval: 60 seconds
```

Value:
- Early warning of attacks
- Reduced false positives (AI filtering)
- Clear remediation guidance
- Reduced MTTR (Mean Time To Respond)

2. Capacity Planning

Scenario: Determine when to scale KDC infrastructure

Setup:

```
# Long baseline for trend analysis
--baseline-hours 168  # 1 week

# Store all data for historical analysis
```

Value:
- Trend analysis shows growth
- Predict when resources will be exhausted
- Justify infrastructure investment
- Proactive scaling before issues

3. Incident Investigation

Scenario: Understand what happened during security incident

Setup:

```
# Query historical database
sqlite3 performance_monitoring/metrics.db

# Review anomaly reports from incident time
ls -l performance_monitoring/anomalies_*
```

Value:
- Timeline reconstruction
- Root cause identification
- Impact assessment
- Lessons learned documentation

4. Performance Optimization

Scenario: Improve KDC response times

Setup:

```
# Detailed monitoring
--interval 30

# Focus on response time metrics
```

Value:
- Identify performance bottlenecks
- A/B test configuration changes
- Measure optimization impact
- Document performance improvements

🚨 Troubleshooting

"KDC process not found"

Cause: Script can't find krb5kdc process
Solution:
1. Verify KDC is running: `ps aux | grep krb5kdc`
2. Check process name variations
3. Run with sudo if permission issue

"Cannot read KDC logs"

Cause: Permission denied on /var/log/krb5kdc.log
Solution:
1. Run with sudo
2. Or add user to appropriate group
3. Or modify log file permissions

"Insufficient data for baseline"

Cause: Less than configured hours of historical data
Solution:

1. Wait for baseline period to complete
2. Reduce --baseline-hours temporarily
3. Monitor continues, just no anomaly detection yet

High false positive rate

Cause: Baseline doesn't match current usage patterns
Solution:
1. Reset baseline during normal operation period
2. Increase baseline window (more data = more stable)
3. Adjust critical thresholds if needed

📊 Performance Impact

Resource Usage:
- CPU: ~2-5% additional (mostly during metric collection)
- Memory: ~50-100MB
- Disk: ~10MB per day (metrics database)
- Network: Minimal (local monitoring)

Suitable for production use: Yes, with minimal impact

🎓 Learning Value

This tool demonstrates:

✅ **System Operations**
- Linux process monitoring
- Log parsing
- Database management
- Performance analysis

✅ **Security Operations**
- Anomaly detection
- Baseline establishment
- Threat identification
- Incident response

✅ **Software Engineering**
- Python best practices
- Clean architecture
- Error handling
- Documentation

✅ **AI Integration**
- API usage
- Contextual analysis
- Result interpretation

✅ **DevOps/SRE**
- Monitoring patterns
- Alerting strategies
- Performance tuning
- Capacity planning

🚀 Next Steps

Enhancements to consider:

1. **Real-time dashboard** (web-based monitoring)
2. **Predictive analytics** (ML-based prediction)
3. **Automated remediation** (auto-scale, auto-block)
4. **Multi-KDC support** (monitor entire cluster)
5. **Custom metrics** (organization-specific KPIs)

📚 Additional Resources

- MIT Kerberos Documentation
- KDC Performance Tuning Guide
- Anthropic Claude API Docs

Created by: Emerson
License: MIT
Support: GitHub Issues

This guide is part of the **Enterprise Kerberos Security** project.